

Aula 2 — React Native em Profundidade

"A Shopify saiu do RN. Ainda vale a pena em 2026?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 28/09/2026

Abertura — Fluxo Fork + PR (~15min)

Antes do conteúdo técnico, vamos cobrir **como entregar atividades via GitHub**.

 Slides + guia escrito no repo da disciplina:

- `slides/intro-fluxo-pr-github.pdf`
- `COMO_ENTREGAR.md` (raiz do repo)

Cobre: fork → clone → branch → commit → push → PR → link Canvas.

Termina com **demo ao vivo** num repo dummy.

| Garantir que ninguém trave na entrega da próxima atividade.

Recap da Aula 1

- 4 tipos de apps móveis com trade-offs distintos
- Estudos de caso: Airbnb, Discord, Coinbase, Alibaba, Shopify (saída do RN, set/2026)
- ADRs como artefato de decisão arquitetural
- Setup ambiente RN funcionando

Atividade 1 — ADR Arquitetural entregue? (espero que sim)

Agenda da aula (3h30)

1. **Bloco 1 (75min)** — Old Architecture vs New Architecture
2. **Intervalo (30min)**
3. **Bloco 2 (75min)** — Hands-on: app de feed com Zustand + TanStack Query
4. **Esquenta (15min)** — Atividade 2

Objetivos de aprendizagem

Ao final da aula você será capaz de:

- **Compreender** Old vs New Architecture do RN (Bridge → JSI/Fabric/TurboModules)
- **Diferenciar** Hermes engine vs JSC e quando cada um faz sentido
- **Aplicar** React Navigation v7 com stack, tabs e deep linking
- **Implementar** TanStack Query com cache invalidation + Zustand store global
- **Desenvolver** animações performáticas com Reanimated v3 worklets

Por que RN ainda gera dúvida?

| "Se a Shopify saiu do RN (set/2026), por que eu continuaria?"

A dúvida de 2026 não é mais performance — é estratégica. "RN é lento?" foi a pergunta de 2018-2021 (era do Bridge); hoje é resolvida. A pergunta real é "RN vs nativo, RN vs Flutter, vale o trade-off pro meu produto?" — e isso depende do contexto, não é resposta única.

Ainda assim, entender por que a fama de "lento" persiste ajuda a responder com dado, não com opinião. A New Architecture (oficial 2022+) reescreveu **a comunicação JS↔nativo** do zero:

-  Bridge assíncrono com serialização JSON (lento, ineficiente)
-  JSI (JavaScript Interface) — chamadas síncronas via referência C++
-  Fabric — novo renderer concorrente
-  TurboModules — native modules com Codegen
-  Hermes — bytecode pré-compilado

Old Architecture — como era

JS Thread

React reconciler



Native Thread

UIKit / Android Views



BRIDGE (assíncrono)
Serializa tudo em JSON e envia

Problema: cada chamada vira mensagem

Bottleneck: scroll com many items

Animação: depende de ida-e-volta

New Architecture — como é agora

JS Thread

Native Thread

React reconciler

UIKit / Android Views

JSI — referência direta C++
Chamada síncrona, sem serializar

Fabric: layout roda em thread de background

TurboModules: native modules carregam sob demanda

Resultado: latência cai de ms pra μ s

Mesmas duas pontas (JS Thread, Native Thread) — a caixa do meio que mudou. Sem fila de mensagens JSON, é acesso direto.

New Architecture — JSI

JSI (JavaScript Interface): interface de baixo nível em **C++** que permite chamadas síncronas diretas JS ↔ Native.

⚠ JSI substitui a **bridge antiga** (que era async + JSON serializado). Mesmo nome "JS Interface", conceito oposto — não há mais bridge, há acesso direto via C++.

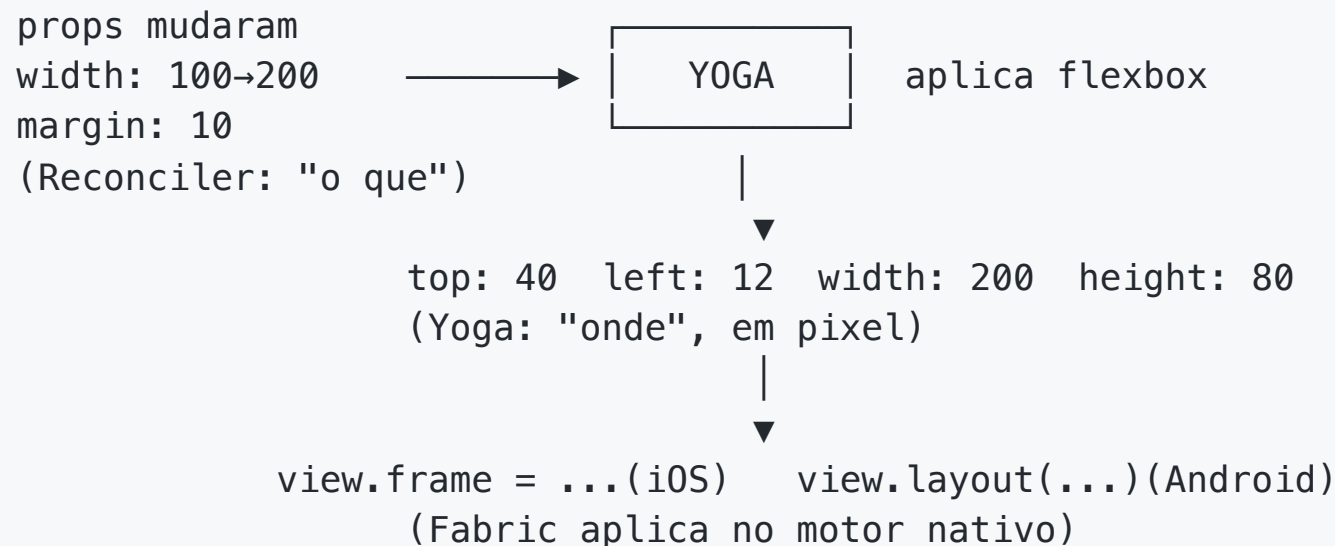
```
// Native side expõe HostObject
class MyTurboModule : public HostObject {
    jsi::Value get(jsi::Runtime& rt, const PropNameID& name) override {
        // ...
    }
};

// JS side acessa via referência direta
import { MyTurboModule } from 'react-native-my-module';
const result = MyTurboModule.getValue();
// SÍNCRONO. Sem JSON. Sem bridge.
```

Resultado: latência de chamada cai de ms para µs.

O gargalo do layout — por que existe o Yoga

UIKit e Android Views não entendem flexbox nativamente. Por isso o RN precisa de um layout engine cross-platform: o **Yoga** (Meta).



Fonte: [Meta Engineering, Yoga](#).

New Architecture — Fabric

Fabric: novo renderer com **commit phase concorrente**.

- Antes: layout + render acoplados em JS thread
- Agora: layout em background, render sincronizado com UI thread
- Suporte a **suspense** e **concurrent mode** do React 18+

Resultado: **menos jank**, especialmente em telas com listas e animações simultâneas.

TurboModules + Codegen — o que é Spec?

Spec = contrato declarativo entre JS e nativo. TS interface descrevendo quais funções o módulo nativo expõe + tipos exatos. **Source of truth** entre 3 plataformas (JS, Android, iOS).

Codegen lê a Spec → gera Kotlin + Swift + JSI bindings automaticamente — em vez de 3 pessoas escreverem 3 versões do mesmo contrato na mão.

Antes do Codegen — 3 lugares, 3 chances de error

```
// TS: quem chama ACHA que isso é verdade  
getValue(arg: string): Promise<number>;
```

```
// Kotlin: escrito à mão, ninguém checa contra o TS  
@ReactMethod  
fun getValue(arg: String?, promise: Promise) { // ← String? , não String!  
    promise.resolve(arg?.length ?: 0) // null vira 0 em silêncio  
}
```

O que acontece: JS manda `undefined` por engano (bug em outro lugar do app). TS não bloqueia em runtime. Kotlin aceita `null` de boa e devolve `0`. **Ninguém percebe — o app não crasha, só devolve o número errado.** Descoberto 2 meses depois, em produção, por um usuário reclamando.

Com `getEnforcing<Spec>()` (próximo slide), essa divergência **derruba o app no boot** — rápido e óbvio, não silencioso 2 meses depois.

TurboModules + Codegen — exemplo

```
// 1. Definir spec em TS — fonte única de verdade
import { TurboModule, TurboModuleRegistry } from 'react-native';

export interface Spec extends TurboModule {
  getValue(arg: string): Promise<number>;
  getArray(): { items: string[] };
}

export default TurboModuleRegistry.getEnforcing<Spec>('MyModule');
// getEnforcing valida na init que módulo nativo implementa o contrato.
// Se faltar getValue/getArray → throw imediato (vs runtime null).
```

```
# 2. Codegen gera Kotlin + Swift bindings
yarn codegen
```

Tipos consistentes entre JS, Android e iOS. Adeus, bugs de `Maybe<string>` virando `null`.

Hermes engine

Hermes é engine JS otimizado para **mobile**, criado pela Meta. Substitui o **JSC** como engine padrão do RN.

JSC = JavaScriptCore. Engine JS da Apple (mesmo do Safari). Era o default do RN antes do Hermes. iOS sempre usou JSC (Apple proíbe JIT de terceiros); Android embarcava JSC junto pra ter mesmo engine nos 2 OS.

- Bytecode pré-compilado (`.hbc`) → cold start mais rápido
- Memória sob controle em devices low-end
- GC otimizado pra apps RN
- Padrão em RN 0.70+ (desde 2022)

```
// Verificar Hermes ativo em runtime
console.log('Hermes ativo?', global.HermesInternal !== null);
```

JSC vs Hermes — comparativo

Critério	JSC	Hermes
Carregamento JS	Parse em runtime (lento)	Bytecode pré-compilado
Cold start	baseline	~40-55% mais rápido
GC	Genérico (mais pausas)	Otimizado pra mobile
Tamanho binário	Maior	~30% menor
iOS	Único permitido (sem JIT)	Roda como interpretador
Android	Embarcado pelo Facebook	JIT permitido (ganho maior)
Status RN 2026	Legado (default até 0.69)	Default desde 0.70

Ganho de cold start: [Callstack, Hermes Performance on iOS](#) — benchmarks reais variam 40-63% conforme app/device, não é número fixo.

JSC vs Hermes — por que o GC importa

GC = Garbage Collection (coleta automática de memória JS). Roda em pausas **stop-the-world** — a JS thread trava durante a pausa.

Mobile tem frame budget de **16ms** (60fps). Pausa de GC maior que isso = **jank visível** pro usuário.

- **JSC GC** veio do Safari — genérico, não pensado pra mobile
- **Hermes GC** foi escrito especificamente pra apps RN — pausas menores e mais previsíveis

New Architecture (Fabric, TurboModules) requer Hermes na prática. JSC fica só como compatibilidade legada.

React Navigation v7 — stack + tabs

```
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';

const Stack = createNativeStackNavigator();
const Tabs = createBottomTabNavigator();

function RootStack() {
  return (
    <Stack.Navigator>
      <Stack.Screen name="Home" component={HomeTabs} />
      <Stack.Screen name="Detail" component={DetailScreen} />
    </Stack.Navigator>
  );
}
```

`createNativeStackNavigator` usa **navigation nativa** (UINavigationController iOS, Fragment Android), não emula em JS.

React Navigation — Deep linking

```
const linking = {
  prefixes: ['myapp://', 'https://myapp.com'],
  config: {
    screens: {
      Home: '',
      Detail: 'item/:id',
      Profile: 'user/:username',
    },
  },
};

<NavigationContainer linking={linking}>
  {/* ... */}
</NavigationContainer>
```

Deep links funcionam:

- iOS: Universal Links + Custom URL schemes
- Android: App Links + Intent filters

Estado em RN 2026 — separar client state de server state

Tipo	Exemplo	Ferramenta 2026
Client state	counter, theme, modal aberto, favoritos locais	Zustand
Server state	lista de filmes da API, perfil do user, posts	TanStack Query

Regra arquitetural: **match tool to type of state**. Não use Redux pra tudo.

Zustand — store em 1 arquivo

```
import { create } from 'zustand';

type CounterState = {
  count: number;
  increment: () => void;
  decrement: () => void;
  reset: () => void;
};

export const useCounterStore = create<CounterState>((set) => ({
  count: 0,
  increment: () => set((s) => ({ count: s.count + 1 })),
  decrement: () => set((s) => ({ count: s.count - 1 })),
  reset: () => set({ count: 0 }),
}));
```

Sem Provider, sem configureStore. Hook usado direto em qualquer componente.

Zustand — uso em componente

```
function HomeScreen() {  
  const { count, increment, reset } = useCounterStore();  
  
  return (  
    <View>  
      <Text>{count}</Text>  
      <Button title="+1" onPress={increment} />  
      <Button title="Reset" onPress={reset} />  
    </View>  
  );  
}  
  
// Seletor pra evitar re-render desnecessário:  
const count = useCounterStore((s) => s.count);
```

Por baixo: `useSyncExternalStore`. Store é módulo singleton fora da árvore React.

TanStack Query — setup

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';

const queryClient = new QueryClient();

export default function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <NavigationContainer>{/* ... */}</NavigationContainer>
    </QueryClientProvider>
  );
}
```

1 Provider no root. QueryClient gerencia cache, dedup, refetch, invalidation.

TanStack Query — useQuery

```
import { useQuery } from '@tanstack/react-query';

function MovieList() {
  const { data, isLoading, error, refetch } = useQuery({
    queryKey: ['movies', 'popular'],
    queryFn: async () => {
      const res = await fetch('https://api.themoviedb.org/3/movie/popular');
      return res.json();
    },
  });

  if (isLoading) return <ActivityIndicator />;
  if (error) return <Text>Erro</Text>;

  return <FlatList data={data?.results} renderItem={/* ... */} onRefresh={refetch} />;
}
```

`queryKey` = identidade do cache. Cache, retry, dedup, stale time — tudo automático.

TanStack Query — mutation + invalidation

```
import { useMutation, useQueryClient } from '@tanstack/react-query';

function AddMovieButton() {
  const queryClient = useQueryClient();

  const mutation = useMutation({
    mutationFn: (movie) => fetch('/api/movies', {
      method: 'POST',
      body: JSON.stringify(movie),
    }),
    onSuccess: () => {
      // Invalida cache 'movies' → refetch automático
      queryClient.invalidateQueries({ queryKey: ['movies'] });
    },
  });

  return <Button title="Add" onPress={() => mutation.mutate(newMovie)} />;
}
```

Mutation termina → `invalidateQueries` → lista refetch automático. Zero código manual.

Threads no React Native

Thread	Roda	Bloqueio =
Main / UI thread	Nativo Android/iOS. Renderiza pixels, processa touch, layout final	Tela trava (jank)
JS thread	Seu código React (hooks, fetch, handlers). Single-threaded	App lento mas tela ainda anima
Shadow thread	Yoga calcula layout (flexbox)	Layout demora

Comunicação entre threads:

- **Old Architecture:** bridge (async, JSON serializado, lento)
- **New Architecture:** JSI (síncrono, sem JSON, via C++)

Animação tradicional: JS thread calcula style → UI thread renderiza. Se JS thread trava (fetch, parse), animação **engasga**.

Reanimated v3 — worklets

Worklet: função JS que **roda direto na UI thread**, não na JS thread.

Mesmo com JS thread 100% bloqueada, worklet anima a 60/120fps porque não depende dela.

```
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withSpring,
} from 'react-native-reanimated';

function AnimatedBox() {
  const offset = useSharedValue(0);

  const style = useAnimatedStyle(() => {
    return {
      transform: [{ translateX: offset.value }],
    };
  });
  // ... componente continua no próximo slide
}
```

Worklet — diretiva `'worklet'`

Função stand-alone só vira worklet com **diretiva no topo**:

```
// SEM diretiva: roda no JS thread (default)
function semWorklet(msg: string) {
  console.log('JS thread:', msg);
}

// COM diretiva: roda no UI thread
function comWorklet(msg: string) {
  'worklet';
  console.log('UI thread:', msg);
}
```

Babel plugin do Reanimated detecta `'worklet'` no parse → empacota a função pra rodar no contexto JS secundário da UI thread.



Hooks `useAnimatedStyle`, `useAnimatedGestureHandler` etc auto-aplicam a diretiva — não precisa escrever `'worklet'` dentro deles. Só pra funções stand-alone que você passa pra worklets.

Worklet — render + trigger

```
function AnimatedBox() {  
  // ... offset + style do slide anterior  
  
  return (  
    <>  
    <Animated.View style={[styles.box, style]} />  
    <Button  
      onPress={() => (offset.value = withSpring(100))}  
      title="Move"  
    />  
  </>  
  );  
}
```

Trade-off: worklet tem escopo limitado — não pode `useState`, não pode `fetch`, não pode usar variáveis fora dele. Pra "voltar" pro JS thread = `runOnJS(fn)()`.

Worklet — cuidado com `runOnJS`

```
const handleAnimationEnd = () => {  
  // Função normal – roda em JS thread  
  navigation.navigate('Detail');  
};  
  
const style = useAnimatedStyle(() => {  
  if (offset.value >= 100) {  
    runOnJS(handleAnimationEnd)(); // salta de UI thread → JS thread  
  }  
  return {  
    transform: [{ translateX: offset.value }],  
  };  
});
```

`runOnJS` reintroduz overhead. Use só quando precisar — não em hot path.

Estado em RN — comparativo de libs 2026

Lib	Tipo	Quando usar	Bundle	Downloads/sem
Zustand	client state	Default 2026 — apps small-medium	~1KB	~40M
TanStack Query	server state	Default 2026 — data fetching + cache	~13KB	~48M
Jotai	atomic state	Granular subscriptions (fintech, healthtech)	~2.5KB	~4M
Redux Toolkit + RTK Query	client + server	Enterprise (10+ devs, time-travel debug)	~15KB	~22M
MobX	observable	OOP-like, legado	~50KB	~2,7M

Padrão 2026: Zustand (client) + TanStack Query (server). Match tool to type of state.

Fonte: [npm registry](#), semana de 15-21/09/2026 — número muda toda semana, é ordem de grandeza, não valor fixo.

Demo — IA gerando store + query

Prompt para Claude:

Crie:

1. Zustand store `useFavoritesStore` com favorites: number[] + actions add(id), remove(id), clear(). TypeScript tipado.
2. Hook TanStack Query `useMovies` que busca filmes populares da TMDB com queryKey ['movies', 'popular'] e staleTime 5min.

Output em 2 arquivos: store/favoritesStore.ts e api/useMovies.ts.

IA é boa em scaffolding tipado. **Revise sempre** — alucinações de schema comuns.

Intervalo — 30min

Volta às 20h45

Aproveita pra:

- Esticar perna
- Verificar Node 22 + Expo CLI + simulador funcionando
- Tomar café / água

Próximo bloco é hands-on com Zustand + TanStack Query + Reanimated. Já abre terminal + IDE.

Arquitetura profissional — separar responsabilidades

Camada	Responsabilidade
services/	HTTP (axios + interceptors)
queries/	Server state (TanStack Query)
contexts/	Client state global (theme, auth)
store/	Client state local (Zustand)
screens/	UI pura
components/	Reutilizáveis

Screen não conhece axios, endpoint, cache, retry. Ela só consome dados.

TanStack Query não é lib de fetch. É lib de gerenciamento de **estado de servidor**.

TMDB API — gerar sua key (5min)

1. Conta grátis em <https://www.themoviedb.org/signup>
2. **Settings** → **API** → Request API key (Developer, uso pessoal/educacional)
3. Copia o **API Read Access Token** (formato `eyJhbGc...` longo)
4. No starter, `cp .env.example .env` e cola:

```
EXO_PUBLIC_TMDB_TOKEN=eyJhbGc...seu_token_aqui
```

themoviedb.org → Settings

- Edit Profile
- API ← *clica aqui*
- ...

API Read Access Token

`eyJhbGc...` XXXXXXXXXX (copia esse)

Endpoints TMDb que vamos usar

```
# 1. Filmes populares (paginado)
curl 'https://api.themoviedb.org/3/movie/popular?language=pt-BR&page=1' \
  -H "Authorization: Bearer $TOKEN"

# 2. Detalhe de 1 filme (id=603 = The Matrix)
curl 'https://api.themoviedb.org/3/movie/603?language=pt-BR' \
  -H "Authorization: Bearer $TOKEN"

# 3. Buscar por nome
curl 'https://api.themoviedb.org/3/search/movie?query=matrix&language=pt-BR' \
  -H "Authorization: Bearer $TOKEN"
```

`src/services/api.ts` no starter encapsula com axios + interceptor. Você usa via `useQuery` em `src/queries/movies/`.

Hands-on — vamos fazer juntos (~45min)

Setup (~5min) — clone direto do repo do prof

Pra **acompanhar a aula** (sem entregar nada agora):

```
git clone https://github.com/jacksonsmith/puc-iec-mobile-multiplataforma.git
cd puc-iec-mobile-multiplataforma/exercicios/02-app-rn-navegacao-estado/starter
npm install
cp .env.example .env          # editar com seu TMDB token
npx expo start --web
```

⚠ Pra **entregar** Atividade 2 depois: faz **fork** primeiro (botão no GitHub) → clone do **seu fork** → push pro fork → CI valida testes. Detalhes em `COMO_ENTREGAR.md`.

Roteiro de TASKs (hands-on aula)

1. **TASK 1:** preencher `src/store/counterStore.ts` (Zustand counter) — 10min
2. **TASK 2:** preencher `src/queries/movies/get-popular-movies.ts` (TanStack useQuery) — 15min
3. **TASK 3:** preencher `src/screens/MovieList.tsx` (FlatList) — 10min
4. **TASK 4:** gerar testes counter com IA — 5min

Roteiro mestre: `PASSOS.md` na raiz do starter.

TASKs 5-10 → Atividade 2 assíncrona (15pts, entrega ver Canvas).

Como navegar o starter

Cada arquivo tem comentários `// TODO [TASK N]:` indicando o que fazer.

```
# Lista todas tasks pendentes:  
grep -rn "TODO \[TASK" src/ __tests__/  
  
# Quantas faltam:  
grep -c "TODO \[TASK" src/**/*.ts src/**/*.tsx __tests__/*.ts
```

Roteiro mestre: **PASSOS.md** na raiz do starter.



Filosofia: você **não escreve do zero**. Starter tem 90% pronto. Você preenche os 10% que importam — actions, hooks, render. **Foca no conceito, não no boilerplate.**

TASK 1 — Zustand counter store

 Abre `src/store/counterStore.ts` → procura `TODO [TASK 1]`

```
// TODO [TASK 1]: implementar
export const useCounterStore = create<CounterState>((set) => ({
  count: 0,
  // ← preenche increment, decrement, reset
}));
```

Consome em `src/screens/MovieList.tsx`:

```
const count = useCounterStore((s) => s.count);
```



Sem Provider. Sem configureStore. Hook é o próprio store.

TASK 2 — TanStack Query usePopularMovies

 Abre `src/queries/movies/get-popular-movies.ts` → procura `TODO [TASK 2]`

```
// TODO [TASK 2]: substituir stub por useQuery real
export const usePopularMovies = (page = 1) =>
  useQuery({
    queryKey: ['movies', 'popular', page],
    queryFn: () => fetchPopularMovies(page),
    staleTime: 1000 * 60 * 5,
  });
```

`fetchPopularMovies` já existe no arquivo — usa `api` do `src/services/api.ts`.



`queryKey` = identidade do cache. TanStack dedupe + invalidate por essa key.

TASK 3 — FlatList + MovieCard

 Abre `src/screens/MovieList.tsx` → procura `TODO [TASK 3]`

```
const { data, isLoading, error, refetch } = usePopularMovies();

// TODO [TASK 3]: renderizar
<FlatList
  data={data?.results ?? []}
  keyExtractor={(item) => String(item.id)}
  renderItem={({ item }) => <MovieCard movie={item} />}
  onRefresh={refetch}
  refreshing={isLoading}
/>
```

`MovieCard` já está pronto em `src/components/MovieCard.tsx` (sem favoritar — Atividade 2 adiciona via TASK 6).

TASK 4 — IA pra escalar testes

Starter já vem com **Jest configurado** + 1 teste exemplo em `__tests__/counterStore.test.ts`.

```
// Atual:
test('increment aumenta count em 1', () => {
  useCounterStore.getState().increment?();
  expect(useCounterStore.getState().count).toBe(1);
});

// TODO [TASK 4]: gerar +2 testes com IA
```

Prompt sugerido pra Claude/Cursor:

"Adicione testes Jest pra useCounterStore cobrindo decrement, reset, e 100 increments seguidos. Mantenha estilo dos testes existentes."

💡 Testes = onde IA mais agrega. Boilerplate puro, baixo risco.

CI do GitHub Actions valida **≥ 6 testes verdes** (3 counter TASK 4 + 3 favorites TASK 9). Sem CI verde = perde pontos na Atividade 2.

Atividade 2 — TASKs 5 a 10

Após hands-on, você tem TASKs 1-4 prontas. Atividade 2 (15pts, entrega ver Canvas) adiciona:

TASK	O que faz
TASK 5	Zustand <code>useFavoritesStore</code> (toggle, isFavorite, add, remove, clear)
TASK 6	Integrar favorites em <code>MovieCard</code> (botão  funcional)
TASK 7	Persistência MMKV (favoritos sobrevivem a reload)
TASK 8	<code>HeartButton</code> com <code>Reanimated</code> (escolha A heart pop / B swipe / C shared element)
TASK 9	Testes favorites com IA (≥ 3 testes verdes)
TASK 10	README + screencast.gif + push pro fork → CI verde

Detalhes em `exercicios/02-app-rn-navegacao-estado/enunciado.md` + `PASSOS.md` do starter.

Pitfall 1 — `useEffect` pra data fetching

```
// ❌ ERRADO
function MovieList() {
  const [movies, setMovies] = useState([]);
  useEffect(() => {
    fetch('/api/movies').then((r) => r.json()).then(setMovies);
  }, []);
}
```

Race conditions, sem cache, sem dedup, refetch toda navegação.

```
// ✅ CERTO — TanStack Query
const { data } = useQuery({
  queryKey: ['movies'],
  queryFn: () => fetch('/api/movies').then((r) => r.json()),
});
```

Pitfall 2 — Worklet sem diretiva `'worklet'`

```
// ❌ ERRADO — roda no JS thread mesmo passando pra worklet
function log(msg) {
  console.log(msg);
}

const style = useAnimatedStyle(() => {
  log('frame'); // → erro: "log is not a worklet"
  return { /* ... */ };
});
```

```
// ✅ CERTO — diretiva torna função worklet
function log(msg: string) {
  'worklet';
  console.log(msg);
}
```

Hooks como `useAnimatedStyle` auto-aplicam. Mas funções stand-alone precisam da diretiva.

Pitfall 3 — `runOnJS` em hot path

```
// ❌ ERRADO — chama runOnJS a cada frame
const style = useAnimatedStyle(() => {
  runOnJS(updateReactState)(offset.value); // 60x/seg → JS thread sobrecarrega
  return { transform: [{ translateX: offset.value }] };
});
```

```
// ✅ CERTO — runOnJS só em eventos (threshold, fim de gesture)
const style = useAnimatedStyle(() => {
  if (offset.value > 200) {
    runOnJS(handleSwipeComplete)(); // 1x quando passar threshold
  }
  return { transform: [{ translateX: offset.value }] };
});
```

`runOnJS` reintroduz a ponte JS thread ↔ UI thread. Use só em eventos discretos.

Pitfall 4 — `useState` global vs Zustand

```
// ❌ ERRADO – prop drilling 4 níveis
function App() {
  const [theme, setTheme] = useState('light');
  return <Nav theme={theme} setTheme={setTheme} />;
}
// Nav → Screen → Component → Button (cada um passa prop)
```

```
// ✅ CERTO – Zustand acessível em qualquer nível
const useThemeStore = create((set) => ({
  theme: 'light',
  toggle: () => set((s) => ({ theme: s.theme === 'light' ? 'dark' : 'light' })),
}));

function Button() {
  const toggle = useThemeStore((s) => s.toggle); // direto
}
```

Pitfall 5 — Misturar client e server state

```
// ❌ ERRADO – server state dentro de Zustand
const useStore = create((set) => ({
  movies: [],           // ← server state (vem de API)
  favorites: [],        // ← client state
  fetchMovies: async () => {
    set({ movies: await fetch(...) }); // refaz cache manual
  },
}));
```

```
// ✅ CERTO – separados
// server state → TanStack Query (cache, dedup, refetch automático)
const { data: movies } = useQuery({ queryKey: ['movies'], queryFn: fetchMovies });

// client state → Zustand (favoritos, toggle local)
const favorites = useFavoritesStore((s) => s.ids);
```

Pitfall 6 — FlatList sem **keyExtractor**

```
// ❌ ERRADO — fallback usa index. Anima errado ao reordenar/remover item.
<FlatList data={movies} renderItem={renderMovie} />

// ❌ ERRADO 2 — inline style em renderItem recria objeto cada render
<FlatList
  data={movies}
  renderItem={({ item }) => (
    <View style={{ padding: 12, margin: 4 }}> {/* novo objeto cada render */}
    <Text>{item.title}</Text>
  </View>
)}
/>
```

```
// ✅ CERTO
const styles = StyleSheet.create({
  card: { padding: 12, margin: 4 }, // referência estável
});
```

```
<FlatList
  data={movies}
  keyExtractor={(item) => String(item.id)}
  renderItem={({ item }) => <View style={styles.card}> {item.title}</View>}
```

Atividade assíncrona — Atividade 2: App RN (15 pts)

Entrega: ver Canvas.

Estender app da aula com:

1. **Tela de favoritos** persistente em MMKV
2. **Animação Reanimated não trivial** (gesture-driven ou shared element)
3. **Medição de TTI** via Flipper / React DevTools profiler
4. **Screencast 2min** mostrando golden path e edge cases

Critérios detalhados de avaliação: Canvas (módulo Atividade 2).

Leituras obrigatórias (pré-Aula 3)

1. **React Native New Architecture WG** (Meta) — visão geral oficial · ~30min

<https://github.com/reactwg/react-native-new-architecture>

2. **Hermes engine — Design doc** (Meta) — bytecode, GC, internals · ~20min

<https://github.com/facebook/hermes/blob/main/doc/Design.md>

3. **Software Mansion blog** — série Reanimated 3 internals · ~25min

<https://blog.swmansion.com/>

4. **React Native — Architecture overview** (docs oficiais) · ~15min

<https://reactnative.dev/architecture/overview>

| Total ~1h30. Quem não ler chega despreparado pro bridging hands-on da Aula 3.

Opcional

Próxima aula (05/10)

Aula 3 — Flutter a fundo: UI, Estado & Isolates

Vamos construir UI + estado (Riverpod) em **Flutter** em sala.

Pré-requisito:

-  **Atividade 2 entregue** (ver Canvas)
-  **Flutter SDK** instalado + emulador Android (ou Xcode/simulador iOS) funcional
-  Ler post-mortem Airbnb *Sunsetting React Native* — 5 partes

Obrigado!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith

Próxima segunda, 05/10, 19:00